

Dynamical Variational Autoencoders for speech generation

Deodato V. Bastos Neto, Valentin Smague, Jean Lou Valeau

MVA 2025/2026 - December 17th, 2025

1 Introduction

Our work is primarily based on the paper *Dynamical Variational Autoencoders: A Comprehensive Review* by Girin et al.[1]. This paper provides an extensive overview of Dynamical Variational Autoencoder (DVAE) models, discussing their theoretical foundations, different architectural choices, and practical training methods. Building upon this comprehensive survey, we have chosen to focus specifically on Variational Recurrent Neural Networks (VRNNs) and Stochastic Recurrent Neural Networks (SRNNs).

Our contribution consists of implementing and training these models using different approaches, including KL annealing and Student’s t priors, to generate speech sequences. We conducted our experiments on the VCTK speech dataset [2], collected by the University of Edinburgh, which we preprocessed into mel-spectrogram representations.

2 Variational Recurrent Neural Networks (VRNNs)

The VRNN combines a recurrent neural network (RNN) with a variational autoencoder (VAE) at each time step. The VAE defines how observations x_t and latent variables z_t are produced sequentially over time, while at each time step t , the VRNN updates its hidden state h_t using a deterministic RNN function d_h :

$$h_t = d_h(\phi_x(x_{t-1}), \phi_z(z_{t-1}), h_{t-1}), \quad (1)$$

where ϕ_x and ϕ_z are feature extractors (multilayer perceptrons in our framework) for the observations and latent variables, respectively, as suggested by Chung et al.[3]. Figure 1a shows the graphical model of a VRNN.

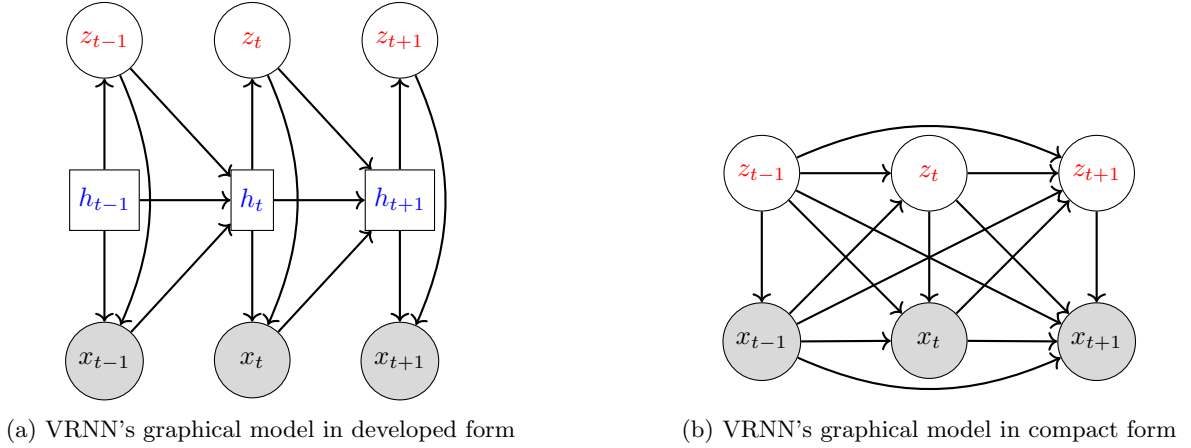


Figure 1: Comparison of VRNN’s graphical models with and without an explicit hidden state.

Using the “unfolding the recurrence” trick, the hidden state h_t can be seen as a function of all previous observations and latent variables and we can rewrite: $h_t = f(x_{t-1}, z_{t-1}, h_{t-1}) = f(x_{1:t-1}, z_{1:t-1})$, where f is deterministic and only serves to represent the past. Hence, we can get rid of h_t in the joint distribution of observations and latents for a sequence of length T , which factorizes as:

$$p_\theta(x_{1:T}, z_{1:T}) = \prod_{t=1}^T p_\theta(x_t \mid z_t, h_t(x_{1:t-1}, z_{1:t-1})) p_\theta(z_t \mid h_t(x_{1:t-1}, z_{1:t-1})) \quad (2)$$

$$= \prod_{t=1}^T p_\theta(x_t \mid x_{1:t-1}, z_{1:t}) p_\theta(z_t \mid x_{1:t-1}, z_{1:t-1}), \quad (3)$$

This factorization is consistent with the compact graphical representation of the VRNN shown in Figure 1b. Now, we can express the log-likelihood of a sequence $x_{1:T}$ as:

$$\begin{aligned}\log p_\theta(x_{1:T}) &= \mathbb{E}_{q(z_{1:T}|x_{1:T})} \left[\log p_\theta(x_{1:T}, z_{1:T}) - \log p_\theta(z_{1:T} | x_{1:T}) \right] \\ &= \mathbb{E}_{q(z_{1:T}|x_{1:T})} \left[\sum_{t=1}^T \log p_\theta(x_t | x_{1:t-1}, z_{1:t}) + \sum_{t=1}^T \log p_\theta(z_t | x_{1:t-1}, z_{1:t-1}) - \log p_\theta(z_{1:T} | x_{1:T}) \right].\end{aligned}$$

where $q(z_{1:T} | x_{1:T})$ is any distribution. Although the following identity holds for any distribution $q(z_{1:T} | x_{1:T})$, it involves the true posterior $p_\theta(z_{1:T} | x_{1:T})$, which is intractable in practice. Thus, we introduce a variational approximation $q_\phi(z_{1:T} | x_{1:T})$ and optimize a lower bound on the log-likelihood. The variational lower bound (VLB) is then defined as:

$$\begin{aligned}\mathcal{L}(\theta, \phi; x_{1:T}) &= \mathbb{E}_{q_\phi(z_{1:T}|x_{1:T})} \left[\log p_\theta(x_{1:T}, z_{1:T}) - \log q_\phi(z_{1:T} | x_{1:T}) \right] \\ &= \mathbb{E}_{q_\phi(z_{1:T}|x_{1:T})} \left[\sum_{t=1}^T \log p_\theta(x_t | x_{1:t-1}, z_{1:t}) + \sum_{t=1}^T \log p_\theta(z_t | x_{1:t-1}, z_{1:t-1}) - \log q_\phi(z_{1:T} | x_{1:T}) \right].\end{aligned}$$

We therefore choose a temporally factorized variational posterior of the form $q_\phi(z_{1:T} | x_{1:T}) = \prod_{t=1}^T q_\phi(z_t | x_{1:t}, z_{1:t-1})$, enabling a decomposition of the variational lower bound into a sum of per-time-step reconstruction and regularization terms:

$$\begin{aligned}\mathcal{L}(\theta, \phi; x_{1:T}) &= \sum_{t=1}^T \mathbb{E}_{q_\phi(z_{1:t}|x_{1:t})} \left[\log p_\theta(x_t | x_{1:t-1}, z_{1:t}) \right] \\ &\quad - \sum_{t=1}^T \mathbb{E}_{q_\phi(z_{1:t-1}|x_{1:t})} \left[\text{KL}(q_\phi(z_t | x_{1:t}, z_{1:t-1}) \| p_\theta(z_t | x_{1:t-1}, z_{1:t-1})) \right].\end{aligned}$$

The first term corresponds to the reconstruction of x_t given past observations and latent variables. This expectation is intractable and is typically approximated using a Monte Carlo sample with the *reparameterization trick*:

$$z_t = \mu_\phi(x_{1:t}, z_{1:t-1}) + \sigma_\phi(x_{1:t}, z_{1:t-1}) \odot \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I), \quad (4)$$

allowing gradients to propagate through the stochastic latent variables.

The KL divergence term can be computed analytically for diagonal Gaussian distributions as it is our case, and is shown in the appendix A. Note that any additive constant independent of the parameters (e.g., $\frac{1}{2} \log 2\pi$) is omitted, since it does not affect the gradients and therefore can be safely ignored during training. As observed in sequential VAEs, the KL term may collapse to zero, causing the latent variable to be ignored by the decoder [4]. To mitigate this effect, we introduce a β coefficient that gradually increases from 0 to 1 during training (KL annealing):

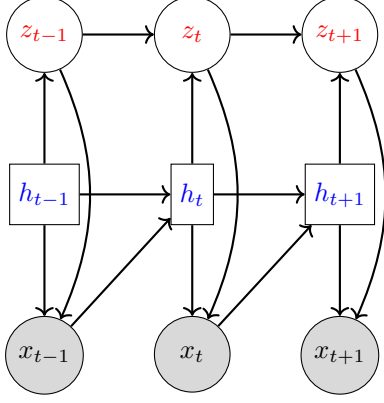
$$\mathcal{L}_\beta(\theta, \phi; x_{1:T}) = \sum_{t=1}^T \log p_\theta(x_t | x_{1:t-1}, z_{1:t}) - \sum_{t=1}^T \beta_t \text{KL}(q_\phi(z_t | x_{1:t}, z_{1:t-1}) \| p_\theta(z_t | x_{1:t-1}, z_{1:t-1})). \quad (5)$$

The training procedure consists of sampling z_t using the reparameterization trick, computing the reconstruction and KL terms, applying β -annealing to the KL, and minimizing the loss function which is the negative VLB.

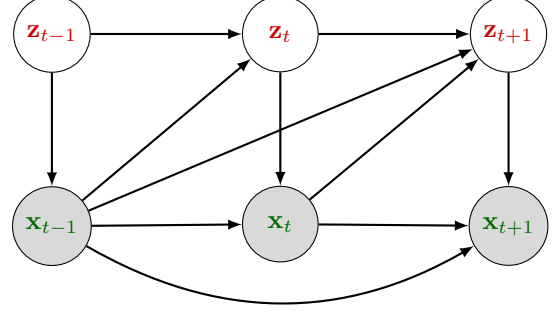
After training the model, one can generate new samples with the following procedure presented in Appendix B. The detailed expressions for the generative model, inference network, feature extractors, and other associated functions used in our implementation are also provided in Appendix A.

3 Stochastic Recurrent Neural Networks (SRNNs)

The SRNN combines a deterministic RNN with a stochastic State Space Model (SSM). The SSM defines the sequential production of x_t and z_t over time. At each time step, the SRNN updates h_t using the deterministic RNN function d_h so we have $h_t = d_h(x_{t-1}, h_{t-1})$.



(a) SRNN's graphical model in developed form



(b) SRNN's graphical model in compact form

Figure 2: Comparison of the SRNN graphical model in its developed form (left), where the deterministic state $\mathbf{h}_t$ is explicit, and compact form (right), where dependencies are marginalized over $\mathbf{h}_t$.

Figure 2a shows the graphical model of an SRNN. As in the VRNN case, the hidden state h_t can be considered a function of all previous observations: $h_t = f(x_{t-1}, h_{t-1}) = f(x_{1:t-1}, h_{1:t-1})$, where f is a deterministic function. Therefore, we can marginalize the joint distribution of all variables w.r.t the hidden state $p_\theta(x_{1:T}, z_{1:T}) = \prod_{t=1}^T p_\theta(x_t | x_{1:t-1}, z_t) p_\theta(z_t | x_{1:t-1}, z_{t-1})$. This factorization is precisely what we can see in Figure 2b, where x_t depends on all the previous observations, $x_{1:t-1}$, as well as the current latent variable, z_t . Meanwhile, z_t depends on all the previous observations as well as the previous latent variable z_{t-1} .

Unlike the VRNN, the SRNN has more dependencies due to the generative model of the observable data used to calculate the exact posterior distribution:

$$p_\theta(z_{1:T} | x_{1:T}) = \prod_{t=1}^T p_\theta(z_t | z_{t-1}, x_{1:T}) \quad (6)$$

To account for this, the approximate posterior distribution, q_ϕ , also depends on past and future observations, which are handled by a backward recurrent neural network (RNN), $e_{\overleftarrow{g}}$, with an internal state, $\overleftarrow{g}_t$. Formally, q_ϕ can be written as follows:

$$q_\phi(z_{1:T} | x_{1:T}) = \prod_{t=1}^T q_\phi(z_t | z_{t-1}, \overleftarrow{g}_t), \quad (7)$$

where $\overleftarrow{g}_t = e_{\overleftarrow{g}}([h_t, x_t], \overleftarrow{g}_{t+1})$.

With that, we obtain a different version of the VLB compared to the VRNN. In this version, x_t depends on z_t , and z_t depends on $x_{1:T}$ and z_{t-1} :

$$\begin{aligned} \mathcal{L}(\theta, \phi; x_{1:T}) &= \sum_{t=1}^T \mathbb{E}_{q_\phi(z_t | x_{1:t})} \left[\log p_\theta(x_t | x_{1:t-1}, z_t) \right] \\ &\quad - \sum_{t=1}^T \mathbb{E}_{q_\phi(z_{t-1} | x_{1:T})} \left[\text{KL}(q_\phi(z_t | x_{1:T}, z_{t-1}) \| p_\theta(z_t | x_{1:t-1}, z_{t-1})) \right]. \end{aligned}$$

Similar to the VRNN, the KL divergence can be computed analytically in the KL divergence, and intractable expectations can be approximated by Monte Carlo estimates.

4 Experiments

VRNN's training

To train our VRNN, the audio signals were first preprocessed into log-mel spectrograms, as this representation is well known to provide more stable and efficient training for neural models operating on speech signals. Then

we launched several trainings, tuning several hyperparameters, including the learning rate and its scheduler, as well as the KL annealing parameter, defined as the number of epochs required for the KL weight to reach its maximum value of 1. After multiple experiments, the best configuration was obtained with 200 training epochs, a KL annealing period of 30 epochs, an initial learning rate of 10^{-3} , and a ReduceLROnPlateau scheduler allowing the learning rate to be gradually reduced down to 10^{-5} .

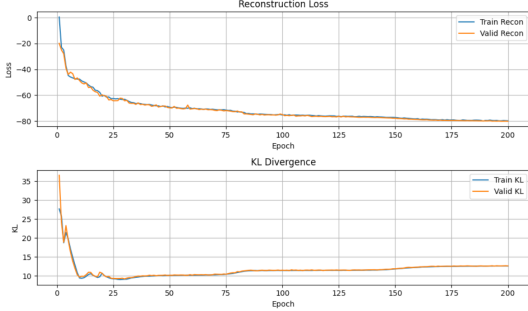


Figure 3: Training and validation reconstruction (top) and KL (bottom) losses during VRNN training.

A comparison between the original audio samples of the test set and their reconstructions is presented in figure 4 and in Appendix C.1.

The main time–frequency structures are well preserved, including the temporal alignment of speech segments and the dominant low-frequency energy corresponding to voiced components. In particular, the model successfully captures the global spectral envelopes and the harmonic patterns that characterize speech, indicating that the VRNN has learned a meaningful representation of the underlying acoustic dynamics.

However, the reconstructed spectrograms appear noticeably smoother than the originals, meaning that fine-grained spectral details are partially attenuated. This smoothing effect is explained by the regularization imposed by the KL divergence, which encourages the model to favor robust and averaged representations over sharp local variations.

We also investigated replacing the standard Gaussian prior with a Student’s t prior with three degrees of freedom. This type of prior is particularly well suited for data exhibiting heavy-tailed distributions, and could potentially lead to improved modeling of speech dynamics. In practice, the training behavior obtained with the Student’s t prior is very similar to the one observed with the Gaussian prior. Both the reconstruction loss and the KL divergence follow comparable trajectories, although the Gaussian prior consistently achieves a slightly lower overall loss. This suggests that, for the considered dataset and architecture, the increased flexibility of the heavy-tailed prior does not translate into a significant performance gain. The evolution of the reconstruction loss and KL divergence, as well as qualitative examples comparing original and reconstructed test samples obtained with the Student’s t prior, are presented in Appendix C.2.

Figure 3 shows the evolution of the reconstruction loss and the KL divergence on both the training and validation sets.

The reconstruction loss converges smoothly, highlighting the stabilizing effect of the learning rate scheduler. Training and validation curves remain closely aligned, indicating good generalization and no overfitting. Finally, the evolution of the KL divergence indicates that the latent space is meaningfully exploited. During the annealing phase (first 30 epochs), the KL term rapidly decreases as the model prioritizes reconstruction. After annealing, the KL divergence gradually increases while the reconstruction loss continues to improve. This behavior suggests that the model progressively refines the posterior distribution, achieving a better balance between reconstruction accuracy and latent regularization, and thus learning a more informative latent representation.

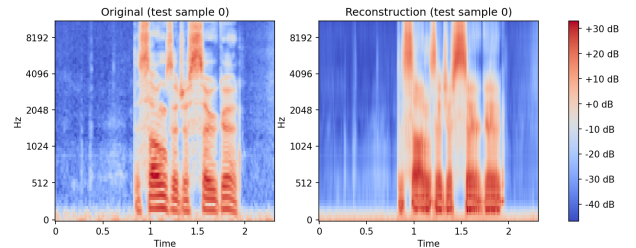
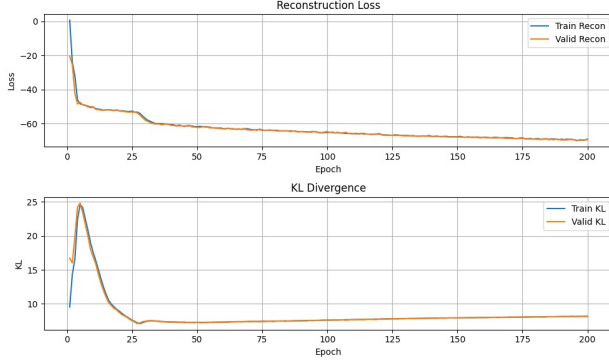


Figure 4: Comparison between original and reconstructed log-mel spectrograms for a test sample.

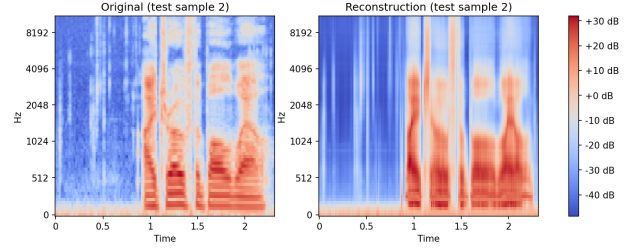
Overall, these results further validate the effectiveness of the proposed VRNN architecture and training strategy, and confirm that the model is well suited for speech generation.

SRNN’s training

For the SRNN training, we use the same dataset and the same methodology. The results of the best-performing model are shown in Figure 5, along with a comparison between an original test sample and its corresponding reconstruction.



(a) Training and validation reconstruction (top) and KL (bottom) losses during SRNN training.



(b) Comparison between original and reconstructed log-mel spectrograms for a test sample.

Figure 5: Training losses and comparison between original and reconstructed speech spectrograms.

The results obtained with the SRNN are highly comparable to those of the VRNN. Although the overall loss achieved by the SRNN is slightly higher, both models exhibit losses with very similar shapes and training dynamics. The reconstruction example illustrates that the SRNN is able to produce high-quality reconstructions, capturing the main temporal and spectral structures of the speech signal. The reconstructed spectrogram preserves the global energy distribution and main speech components, with only minor smoothing effects.

Speech generation with VRNNs

Once the models were trained, we focused on our main objective, which was to generate speech. We conducted this experiment only with the VRNN, as it achieved better performance during training.

When generating spectrograms using the algorithm described in Appendix B, the resulting outputs appear largely random and lack any form of temporal continuity as figure 6a shows. This behavior is clearly visible in the generated spectrograms, which exhibit vertical patterns occurring consecutively without any coherent temporal structure. This is due to both the hidden state h_0 and the initial input x_0 being initialized to zero. Indeed, the model is required to generate speech starting from an empty and unrealistic state, a situation it was never exposed to during training. Consequently, the VRNN has not learned how to properly bootstrap the generation process from such initial conditions, leading to fully random and unstable spectrograms.

To address this issue, we introduce a warm-up phase in the generation process. Instead of initializing the model from zero states, we provide a short segment of real audio in order to initialize meaningful values for the hidden state h_t and the latent variables. Specifically, we feed the model with the first 180 frames of a real spectrogram from the dataset, and then let it generate the continuation by producing 200 additional frames autonomously. Several strategies were explored to improve the quality of the generated audio. We tested different warm-up samples from both the training and test sets, as well as varying warm-up sequence lengths, but these had little effect. Post-processing with a Gaussian smoothing filter reduced high-frequency noise but did not noticeably improve perceptual quality. Finally, we reduced the variance of the latent variable z_t — referred to as the “noise level” in the spectrogram titles of figures 6b, 6c and 6d — to limit stochasticity during generation.

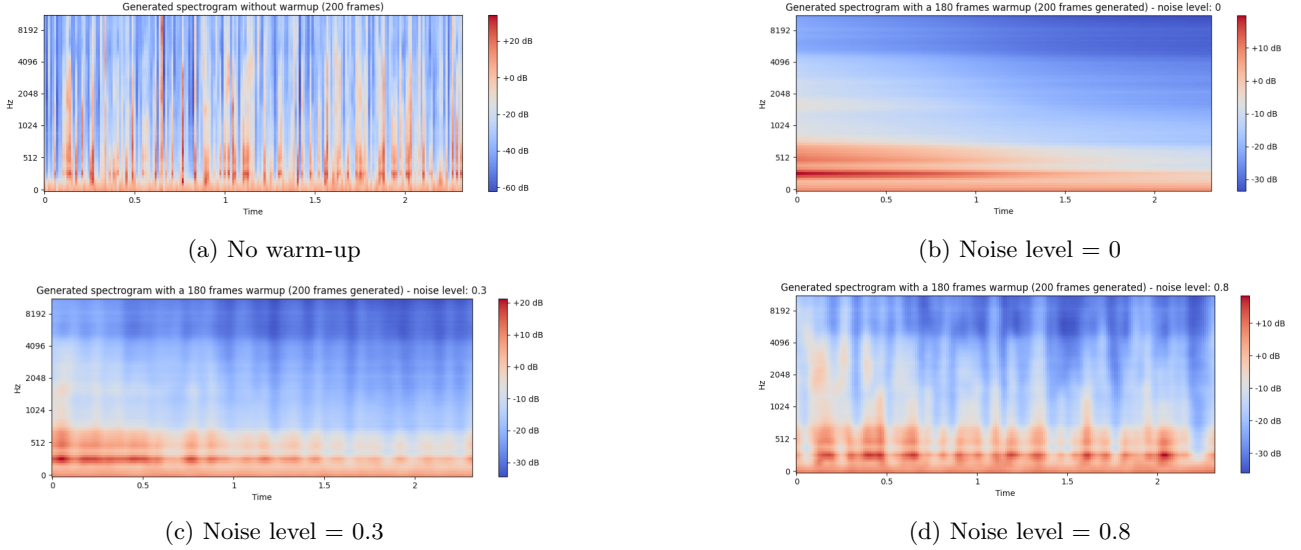


Figure 6: Comparison of four generated spectrograms

Unfortunately, this warm-up strategy alone does not yield perceptually satisfactory results, as the audio obtained from the generated spectrograms is poorly understandable and does not sound truly human. This setting still provides encouraging observations. The model captures a global spectral structure, with low frequencies consistently stronger than high frequencies, reflecting typical speech patterns. Horizontal patterns in the spectrogram indicate that the RNN effectively carries temporal information across frames via the hidden state h_t .

Finally, we trained the model using scheduled sampling, a technique designed to improve generation developed in [5]. Previously, in teacher forcing mode, at each time step t , the model received the true previous frame x_{t-1} to predict x_t . This prevents the model from experiencing errors in past predictions. Consequently, during generation, small prediction errors accumulate over time, leading to meaningless spectrograms and audios—a phenomenon known as exposure bias. Scheduled sampling addresses this by feeding the model either the true frame x_{t-1} with probability ϵ_t or its own previous prediction $\hat{x}_{t-1}$ with probability $1 - \epsilon_t$. We start training with $\epsilon_1 = 1$ and gradually decrease it to 0.5, exposing the model to its own predictions and making it more robust. Here, the relevant metric is the generation loss, computed when the model only uses its own outputs ($\epsilon_t = 0$), rather than the standard validation loss. A low generation loss indicates that the model can produce long, coherent sequences, while a low validation loss only ensures good reconstruction. The results obtained are presented in Appendix C.3. Unfortunately, the audios generated still sound like an old radio.

5 Discussion

In this work, we trained both a VRNN and an SRNN for speech generation. Particular care was taken to avoid KL annealing, as this technique was not described in the original reference paper. The training results demonstrate that both models achieve good performance in reconstructing mel-spectrograms, capturing the main temporal and spectral structures of speech.

Replacing the standard Gaussian prior with a Student’s t prior led to similar results in our experiments, without a clear improvement in reconstruction quality. Overall, the VRNN slightly outperforms the SRNN in terms of loss and reconstruction fidelity. This difference may be explained by the fact that our efforts were primarily focused on optimizing the VRNN, leaving room for further improvements in the SRNN architecture and training procedure.

A notable limitation of our modeling pipeline lies in the conversion of mel-spectrograms back to audio. To generate waveforms, the mel-spectrograms are first converted to STFT representations and then reconstructed using the Griffin–Lim algorithm. This process discards phase information, resulting in audio signals that are intelligible but noticeably robotic. Although the models successfully learn the global structure of speech, the generated audio lacks naturalness and often sounds as if it were produced by an old radio.

As future work, alternative waveform reconstruction methods could be explored, particularly GAN-based vocoders, which have been shown to produce more natural and high-quality audio.

References

- [1] Laurent Girin et al. “Dynamical variational autoencoders: A comprehensive review”. In: (2020). URL: <https://arxiv.org/abs/2008.12595>.
- [2] Christophe Veaux, Junichi Yamagishi, and Simon King. “The Voice Bank Corpus: Design, Collection and Data Analysis of a Large Regional Accent Speech Database”. In: *Proceedings of the 15th Annual Conference of the International Speech Communication Association (INTERSPEECH)*. Lyon, France: International Speech Communication Association, 2013.
- [3] Junyoung Chung et al. “A Recurrent Latent Variable Model for Sequential Data”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 28. 2015. URL: <https://arxiv.org/abs/1506.02216>.
- [4] Samuel R Bowman et al. “Generating sentences from a continuous space”. In: (2015). URL: <https://arxiv.org/abs/1511.06349>.
- [5] Samy Bengio et al. “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems*. 2015.

A VRNN model implementation

We describe our implementation of the VRNN’s generative and inference models, largely following [1]. All models are Gaussian.

Generative Model

The observation model is parameterized as:

$$[\mu_x(z_t, h_t), \sigma_x^2(z_t, h_t)] = d_x(\phi_z(z_t), h_t), \quad (8)$$

$$p_\theta(x_t | z_t, h_t) = \mathcal{N}(x_t; \mu_x(z_t, h_t), \text{diag}(\sigma_x^2(z_t, h_t))). \quad (9)$$

The latent variable prior is given by:

$$[\mu_z(h_t), \sigma_z^2(h_t)] = d_z(h_t), \quad (10)$$

$$p_\theta(z_t | h_t) = \mathcal{N}(z_t; \mu_z(h_t), \text{diag}(\sigma_z^2(h_t))). \quad (11)$$

Inference Model

The inference network is defined as:

$$[\mu_\phi(x_t, h_t), \sigma_\phi(x_t, h_t)] = e_z(\phi_x(x_t), h_t), \quad (12)$$

$$q_\phi(z_t | x_t, h_t) = \mathcal{N}(z_t; \mu_\phi(x_t, h_t), \text{diag}\{\sigma_\phi^2(x_t, h_t)\}). \quad (13)$$

Feature Extraction

The feature extractor for observations, $\phi_x(x_t)$, is implemented as an MLP:

$$\text{MLP}(x_t, 256, \text{Tanh}),$$

and the feature extractor for latents, $\phi_z(z_t)$, as:

$$\text{MLP}(z_t, 32, \text{Tanh}, 64, \text{Tanh}).$$

Recurrent and Output Functions

- **Hidden state update:** $d_h(\phi_x(x_{t-1}), \phi_z(z_{t-1}), h_{t-1})$ is implemented using a GRU network, with input $[\phi_x(x_{t-1}), \phi_z(z_{t-1})]$.
- **Latent prior mapping:** $d_z(h_t)$ maps h_t directly to the latent space dimension through a linear layer.
- **Observation decoding:** $d_x(\phi_z(z_t), h_t)$ is implemented as a simple output layer mapping to the observation space.
- **Inference mapping:** $e_z(\phi_x(x_t), h_t)$ is an output layer taking the concatenation $[\phi_x(x_t), h_t]$ as input.

B Generation Algorithm with a trained VRNN

Algorithm 1 Sequence Generation with a Trained VRNN

Require: Sequence length T , trained VRNN, deterministic flag

1: Initialize hidden state $h_0 \leftarrow 0$

2: **for** $t = 1$ **to** T **do**

3: Compute prior: $(\mu_t, \sigma_t) \leftarrow p_\theta(z_t \mid h_{t-1})$

4: Sample latent:

$$z_t = \begin{cases} \mu_t, & \text{if deterministic} \\ \mu_t + \sigma_t \odot \epsilon, \epsilon \sim \mathcal{N}(0, I) & \end{cases}$$

5: Decode observation: $(\mu_t^x, \sigma_t^x) \leftarrow p_\theta(x_t \mid z_t, h_{t-1})$

6: Sample observation:

$$x_t = \begin{cases} \mu_t^x, & \text{if deterministic} \\ \mu_t^x + \sigma_t^x \odot \eta, \eta \sim \mathcal{N}(0, I) & \end{cases}$$

7: Update hidden state: $h_t \leftarrow d_h(x_t, z_t, h_{t-1})$

8: **end for**

9: **return** $\{x_1, \dots, x_T\}$

C Experiments outputs

C.1 VRNN training with gaussian prior

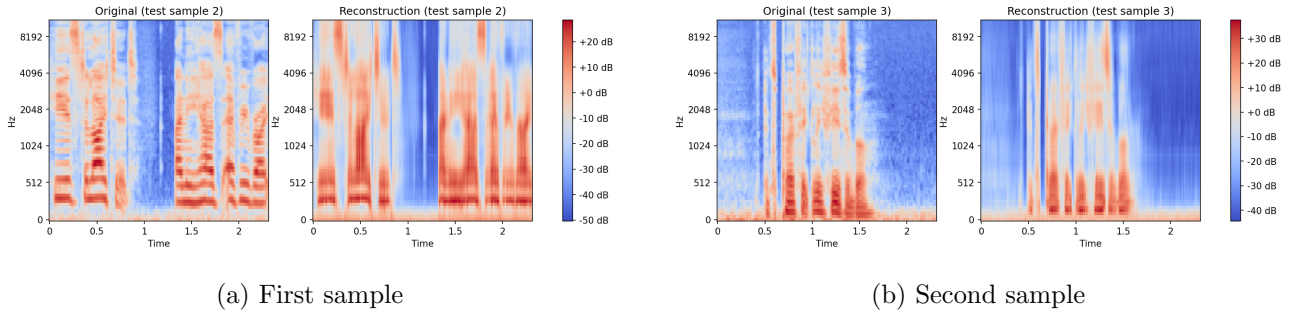


Figure 7: Comparison between original and reconstructed speech spectrograms with gaussian prior.

C.2 VRNN training with student t prior

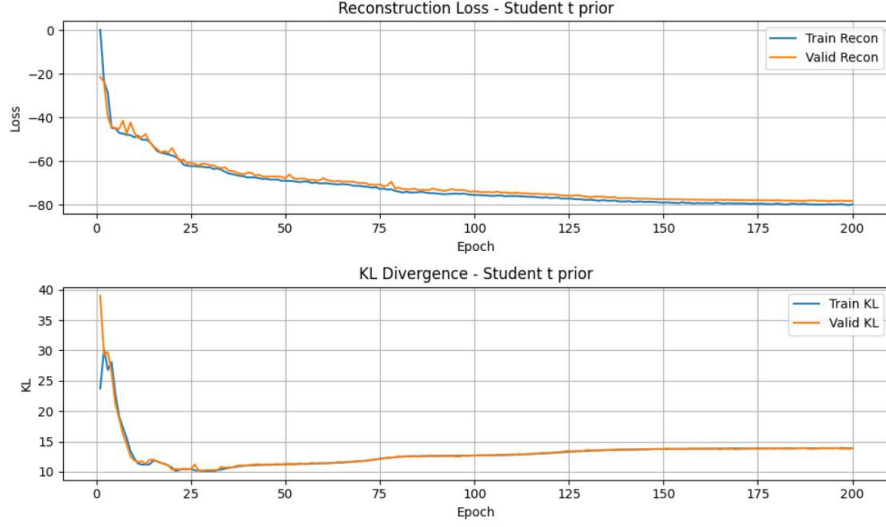


Figure 8: Training and validation reconstruction (top) and KL (bottom) losses during VRNN training with student t prior.

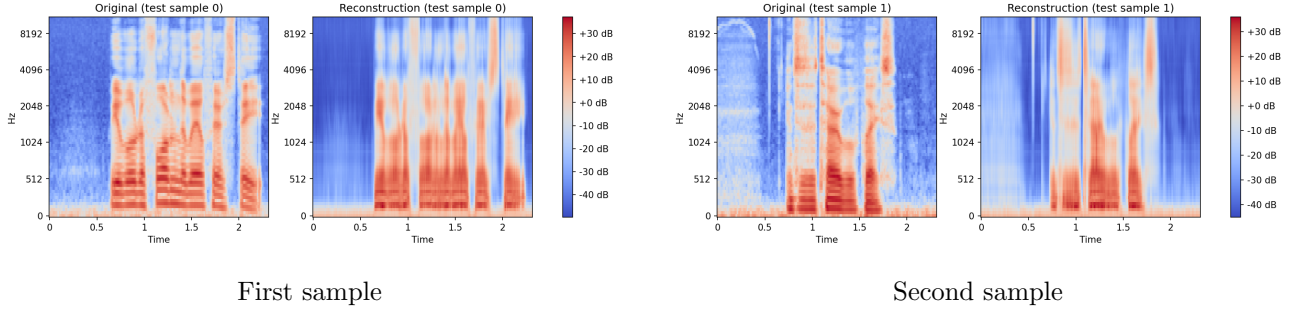
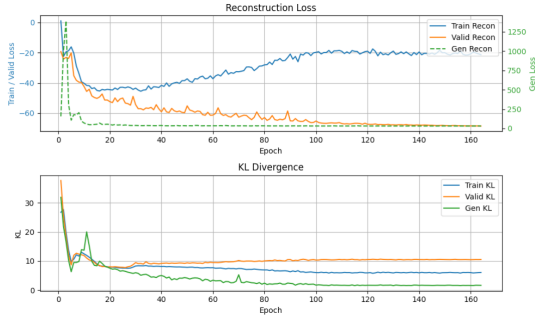


Figure 9: Comparison between original and reconstructed speech spectrograms with student t prior.

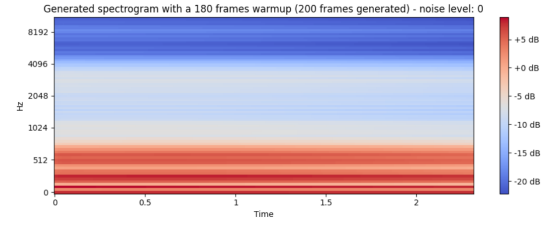
C.3 Speech generation with VRNNs

The blue and yellow lines show loss and KL in teacher forcing mode, while the green line shows them under scheduled sampling. Surprisingly, the training loss exceeds the validation loss in this setting. The generation loss, calculated on the validation set when the model receives $\hat{x}_{t-1}$ instead of x_{t-1} , is on a much larger scale than the standard reconstruction loss, showing that the model struggles to generate without ground truth guidance.

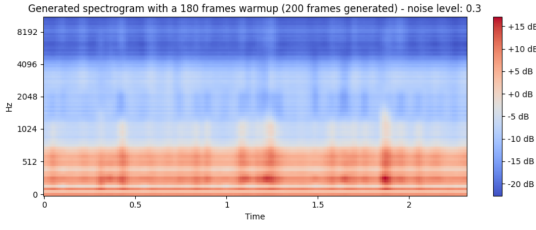
The KL in teacher forcing mode behaves as before, gradually increasing and indicating proper use of the latent space. In contrast, the KL under scheduled sampling decreases toward zero, a phenomenon known as posterior collapse, where the model ignores latent variables. This explains the poor quality of generated audios, which do not improve even when varying the variance of z_t .



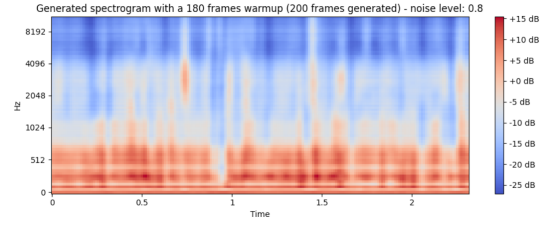
(a) Generation loss



(b) Noise = 0



(c) Noise = 0.3



(d) Noise = 0.8

Figure 10: Generation loss and comparison of three generated samples